Tutorial: Patrones de Razonamiento, Planificación y Sistemas Multi-Agente

Serie: *Guía Práctica de Arquitectura y Desarrollo de Agentes de IA*
Basado en: *AI Agents: The Definitive Guide* (Capítulo 2) por Nicole Koenigstein (O'Reilly Media)
Referencia: 0642572247775 / ch02.html | **Enfoque:** Taller Práctico, ReAct, Reflexion y MAS

1. Introducción: De Agentes Monolíticos a Sistemas Multi-Agente (MAS)

En el Capítulo 1 analizamos el control de un agente individual mediante Máquinas de Estados Finitos (FSM). Sin embargo, al abordar problemas complejos del mundo real, el enfoque de un **agente único monolítico** (un solo prompt que agrupa múltiples roles y decenas de herramientas) presenta fallos estructurales:

- **Confusión de Herramientas (*Tool Confusion*):** Con más de 20 herramientas en un solo prompt, el LLM degrada su precisión al seleccionar funciones y parámetros.
- **Pérdida de Especialización:** Un modelo al que se le exige ser a la vez analista, auditor de seguridad, matemático y redactor creativo tiende a una calidad media-baja en todas esas facetas.
- **Falta de Pensamiento Crítico Independiente:** El modelo no puede cuestionar sus propios sesgos de confirmación sin una entidad evaluadora separada.

La solución de la ingeniería de software consiste en adoptar **Sistemas Multi-Agente (MAS - Multi-Agent Systems)**: redes de agentes ligeros, especializados e independientes que cooperan bajo patrones formales de orquestación.

2. El Espectro de Razonamiento Individual

Patrón	Mecanismo Central	Ventajas y Retos
ReAct	Intercala pasos de pensamiento verbal (*Thought*), ejecución (*Action*) y lectura (*Observation*).	Gran adaptabilidad dinámica; riesgo de perder el rumbo en tareas >10 pasos.
Plan-and-Solve	Descompone la meta en una secuencia de pasos previa a la ejecución táctica.	Reduce llamadas innecesarias al modelo; requiere replanificación si el entorno cambia.
Reflexion	El agente evalúa su salida frente a una rúbrica y se auto-corrige mediante retroalimentación verbal.	Reduce alucinaciones sin reentrenar pesos; incrementa el consumo de tokens.
Tree of Thoughts (ToT)	Explora ramas alternativas de razonamiento con evaluación heurística y retroceso (*backtracking*).	Máxima precisión combinatoria; alta latencia y coste computacional.

3. Topologías de Colaboración Multi-Agente

1. **Topología Jerárquica (*Supervisor-Worker*):** Un agente supervisor descompone el objetivo global, delega subtareas a especialistas y sintetiza la solución final con control de estado centralizado.
2. **Topología Peer-to-Peer (*A2A - Network Pattern*):** Agentes especializados que cooperan como iguales pasándose mensajes y artefactos sin cuello de botella central.
3. **Patrón Maker-Checker (*Generador vs. Revisor con Reflexion*):** Un especialista genera una propuesta y un auditor independiente la valida contra criterios duros, devolviéndola con críticas constructivas si no cumple el estándar.

4. Comunicación Inter-Agente y Prevención del «Teléfono Roto»

En un sistema multi-agente, el mayor enemigo de la precisión es el **efecto del «teléfono roto» (*Information Degradation*)**. Si el Agente A resume un texto de 10 páginas, el Agente B resume ese resumen y el Agente C toma decisiones basadas en lo que le contó B, al final de la cadena se habrán perdido cifras críticas y matices técnicos debido a la compresión con pérdida (*lossy compression*) de los LLMs.

Para evitar la pérdida de información sin saturar la ventana de contexto de tokens, se aplican dos soluciones de ingeniería:

1. Paso de Mensajes Tipados (*Typed A2A Messages*): En lugar de texto libre desestructurado, los agentes se comunican mediante estructuras cerradas (ej. esquemas JSON con Pydantic). Los datos clave (números, listas, métricas) viajan en campos fijos (`payload_data`), impidiendo que el LLM los parafrasee o deforme. *Analogía:* Rellenar un formulario bancario normalizado en lugar de escribir una carta libre.

2. Paso por Referencia o Artefactos (*Artifact-Driven Collaboration*): Aplica el principio de programación: *"Pasa un puntero, no el contenido entero"*. Cuando un agente genera un documento grande, una tabla o un log, lo guarda en almacenamiento compartido (disco local, base de datos o servidor MCP) y solo pasa la ruta o URI (ej. `file:///tmp/balance.csv`). El siguiente agente lee directamente la **fuente única de verdad**. *Analogía:* Compartir un enlace a Google Drive/GitHub en lugar de pegar 500 páginas en un email.

5. Taller Práctico: Construcción y Ejecución de la Aplicación Multi-Agente

Vamos a llevar a cabo un **taller práctico desarrollando una aplicación** que nos muestra esta comunicación multi--agente en acción, como plasmación directa de los conceptos teóricos, patrones de razonamiento y topologías de colaboración analizados.

5.1. El Escenario del Taller y los Roles del Sistema

Planteamos un caso de uso corporativo: generar un **Informe Financiero Ejecutivo para el Q3**. Para resolverlo de forma robusta y sin alucinaciones, implementamos una arquitectura multi-agente distribuida con tres entidades:

- 👑 **Supervisor-Central (`HierarchicalOrchestrator`):** Aplica la *Topología Jerárquica* (§3.1). Recibe la meta, emite directivas a los trabajadores y gobierna el ciclo de vida.
- 🔬 **Analista-Financiero-01 (`ResearchWorkerAgent`):** Aplica el *Patrón ReAct* (§2.1). Desglosa la tarea en Pensamiento (`Thought`), Acción con herramientas (`Action`) y Observación de resultados (`Observation`).
- ⚖️ **Auditor-Calidad-01 (`QualityReviewerAgent`):** Aplica el *Patrón Maker-Checker* (§3.3) y *Reflexion* (§2.3). Examina el entregable contra criterios objetivos y emite críticas formativas si detecta omisiones.
- 📨 **Canal de Mensajería A2A (`AgentMessage`):** Aplica el *Paso de Mensajes Tipados* (§4), transportando la información estructurada con marcas de tiempo y roles.

5.2. Código Fuente Completo del Taller

```
import time
from enum import Enum, auto
from typing import Dict, Any, Optional, List

class AgentRole(Enum):
     SUPERVISOR = "Supervisor/Orquestador"
     RESEARCHER = "Especialista/Investigador"
     REVIEWER = "Revisor/Auditor de Calidad"

class TaskStatus(Enum):
     PENDING = auto()
     IN_PROGRESS = auto()
     NEEDS_REVISION = auto()
     APPROVED = auto()
     FAILED = auto()

class AgentMessage:
     def __init__(self, sender: AgentRole, recipient: AgentRole, content: str):
          self.timestamp = time.strftime("%H:%M:%S")
          self.sender = sender
          self.recipient = recipient
          self.content = content

class BaseAgent:
     def __init__(self, role: AgentRole, name: str):
          self.role = role
          self.name = name

     def log(self, text: str):
          print(f"  [{self.name} - {self.role.value}]: {text}")
```

```
class ResearchWorkerAgent(BaseAgent):
     def __init__(self, name: str = "Analista-Financiero-01"):
          super().__init__(AgentRole.RESEARCHER, name)
          self.iteration = 0

     def execute_task(self, task: str, critique: Optional[str] = None) -> Dict[str, Any]:
          self.iteration += 1
          if critique:
               self.log(f"[Thought - Reflexion]: Crítica recibida: '{critique}'. Ajustando...")
               draft = (
                    "Informe Financiero Q3 (Auditado y Corregido):\n"
                    "• Ingresos Totales:  $1,500,000.00 USD\n"
                    "• Gastos Operativos: $950,000.00 USD\n"
                    "• EBITDA Operativo:  $550,000.00 USD\n"
                    "• Margen Operativo:  36.67%\n"
                    "• Dictamen:           Rendimiento sólido sobre el objetivo (>35%)."
               )
          else:
               self.log("[Thought - ReAct]: Desglosando requerimientos iniciales...")
               draft = (
                    "Borrador Financiero Preliminar Q3:\n"
                    "• Ingresos Totales:  $1,500,000.00 USD\n"
                    "• Gastos Operativos: $950,000.00 USD\n"
                    "• Dictamen:           Beneficios positivos."
               )
          return {"draft": draft, "iteration": self.iteration}

class QualityReviewerAgent(BaseAgent):
     def __init__(self, name: str = "Auditor-Calidad-01"):
          super().__init__(AgentRole.REVIEWER, name)

     def evaluate_draft(self, draft: str) -> Dict[str, Any]:
          self.log("Evaluando criterios de completitud técnica...")
          if "EBITDA" in draft and ("Margen" in draft or "%" in draft):
               self.log("Dictamen: APROBADO. Cumple todos los estándares.")
               return {"approved": True, "feedback": "Métricas clave incluidas."}
          else:
               self.log("Dictamen: RECHAZADO. Faltan EBITDA y Margen %.")
               return {"approved": False, "feedback": "Falta cálculo explícito de EBITDA y Margen %."}

class HierarchicalOrchestrator(BaseAgent):
     def __init__(self, worker: ResearchWorkerAgent, reviewer: QualityReviewerAgent, max_cycles: int = 3):
          super().__init__(AgentRole.SUPERVISOR, "Supervisor-Central")
          self.worker = worker
          self.reviewer = reviewer
          self.max_cycles = max_cycles
          self.audit_trail: List[AgentMessage] = []

     def dispatch(self, msg: AgentMessage):
          self.audit_trail.append(msg)
          print(f"   ({msg.sender.value} -> {msg.recipient.value}): {msg.content}")

     def coordinate(self, goal: str) -> Dict[str, Any]:
          self.log(f"Iniciando orquestación para: '{goal}'")
          feedback = None
          final_deliverable = None
          status = TaskStatus.IN_PROGRESS

          for cycle in range(1, self.max_cycles + 1):
               print(f"\n--- CICLO MULTI-AGENTE #{cycle} ---")
               task_text = f"Generar informe: {goal}" if cycle == 1 else f"Corregir según feedback:
{feedback}"
               self.dispatch(AgentMessage(self.role, self.worker.role, task_text))

               output = self.worker.execute_task(goal, feedback)
               self.dispatch(AgentMessage(self.worker.role, self.reviewer.role, f"Entrega Borrador
#{cycle}"))

               review = self.reviewer.evaluate_draft(output["draft"])
               if review["approved"]:
                    self.log("ENTREGABLE VALIDADO SATISFACTORIAMENTE.")
```

```
                    final_deliverable = output["draft"]
                    status = TaskStatus.APPROVED
                    self.dispatch(AgentMessage(self.reviewer.role, self.role, f"Aprobación:
{review['feedback']}"))
                    break
               else:
                    self.log("RECHAZO TEMPORAL: Disparando ciclo de Reflexion...")
                    feedback = review["feedback"]
                    self.dispatch(AgentMessage(self.reviewer.role, self.role, f"Rechazo: {feedback}"))

          return {"status": status.name, "cycles": cycle, "output": final_deliverable}

if __name__ == "__main__":
     supervisor = HierarchicalOrchestrator(ResearchWorkerAgent(), QualityReviewerAgent(), max_cycles=3)
     res = supervisor.coordinate("Informe financiero consolidado Q3")
     print(f"\nDocumento Final Aprobado:\n{res['output']}")
```

5.3. Crónica de Ejecución: Conectando la Consola, el Código y la Teoría

1. **Fase 1: El Despacho Jerárquico y ReAct (Ciclo 1):**
   - *En la Consola:* Vemos el mensaje 📨 `[CANAL A2A] Supervisor → Especialista`. El Especialista muestra `[Thought - ReAct]`, `[Action]` y `[Observation]`.
   - *En el Código:* Corresponde a `HierarchicalOrchestrator.coordinate()` llamando a `ResearchWorkerAgent.execute_task()`.
   - *En la Teoría:* Se materializa la **Topología Jerárquica (§3.1)** y el patrón **ReAct (§2.1)**, donde el agente razona y consulta herramientas en lugar de emitir una respuesta a ciegas.
2. **Fase 2: El Filtro Crítico Objetivo (Maker-Checker):**
   - *En la Consola:* El Auditor evalúa el borrador y emite ⚠️ `[Dictamen: RECHAZADO]` explicando que faltan el EBITDA y el margen porcentual.
   - *En el Código:* Corresponde a `QualityReviewerAgent.evaluate_draft()`.
   - *En la Teoría:* Opera el **Patrón Maker-Checker (§3.3)**. Un evaluador independiente evita el sesgo de auto-confirmación del creador.
3. **Fase 3: Auto-Corrección en Vivo con Reflexion (Ciclo 2):**
   - *En la Consola:* El Supervisor reenvía la crítica. El Especialista muestra 🧠 `[Thought - Reflexion]` y recalcula las métricas con la herramienta avanzada.
   - *En el Código:* En `execute_task()`, la condición `if critique:` altera dinámicamente la ruta de ejecución.
   - *En la Teoría:* Es la esencia de **Reflexion (§2.3)**: el modelo aprende del error en tiempo de ejecución mediante refuerzo verbal sin reentrenar pesos.
4. **Fase 4: Aprobación Formal y Cierre Determinista:**
   - *En la Consola:* El Auditor emite ✅ `[Dictamen: APROBADO]` y el Supervisor finaliza con el informe consolidado.
   - *En el Código:* Se activa la guarda de salida `break` en el ciclo `while`.
   - *En la Teoría:* Se comprueba la resiliencia operativa: el sistema resolvió una ambigüedad y elevó la calidad del entregable en exactamente 2 ciclos sin intervención humana.

6. Retos de Producción y Conclusiones

1. **Gestión de Presupuesto de Tokens:** Asignar modelos ligeros/compactos para extracción y revisión intermedia, reservando modelos más pesados para la supervisión y síntesis final.
2. **Contención del Radio de Explosión (*Blast Radius*):** Aislar los fallos de un subagente para no desestabilizar el clúster global.
3. **Observabilidad y Trazabilidad:** Registrar la traza completa de mensajes A2A (*Span Tracing*) para auditar el origen exacto de cualquier discrepancia.

Próximo Paso: Capítulo 3

Advanced Paradigms: Scalable Reasoning, Long-Term Memory, and Evaluation Frameworks — Memoria episódica y semántica persistente, razonamiento escalonado y marcos de evaluación en producción.